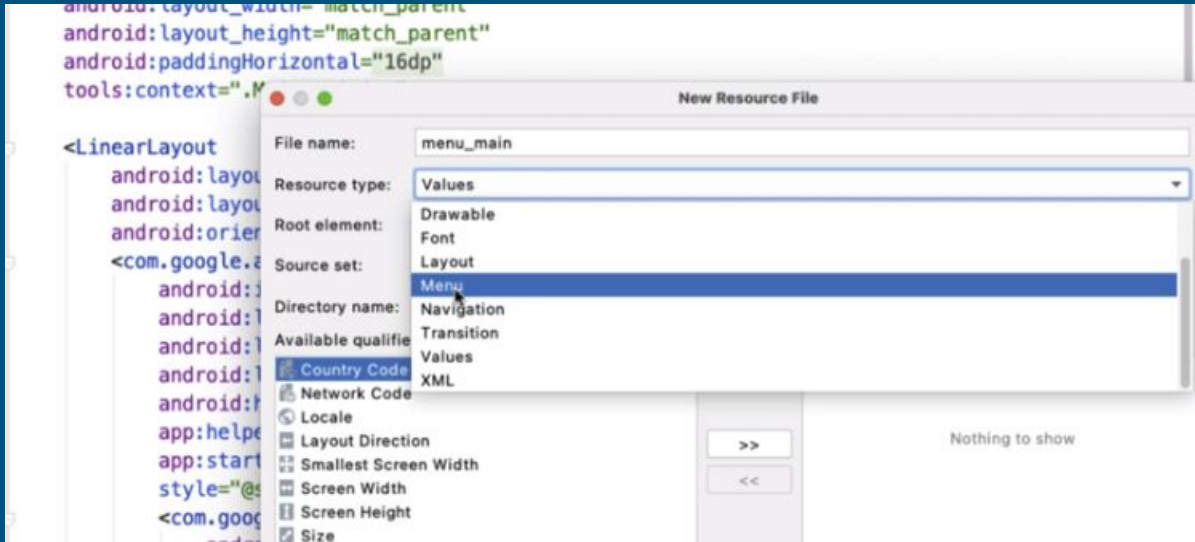
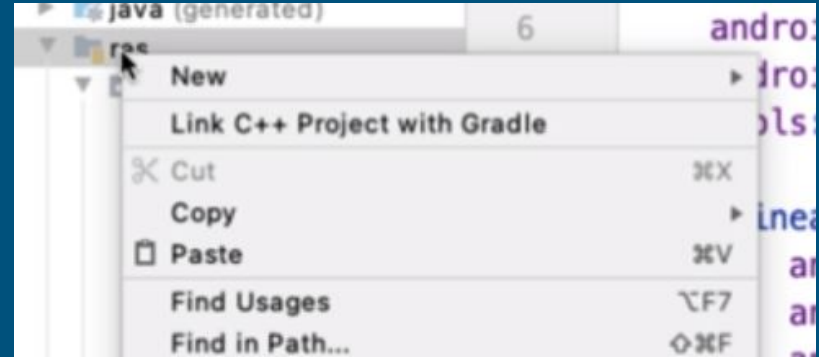
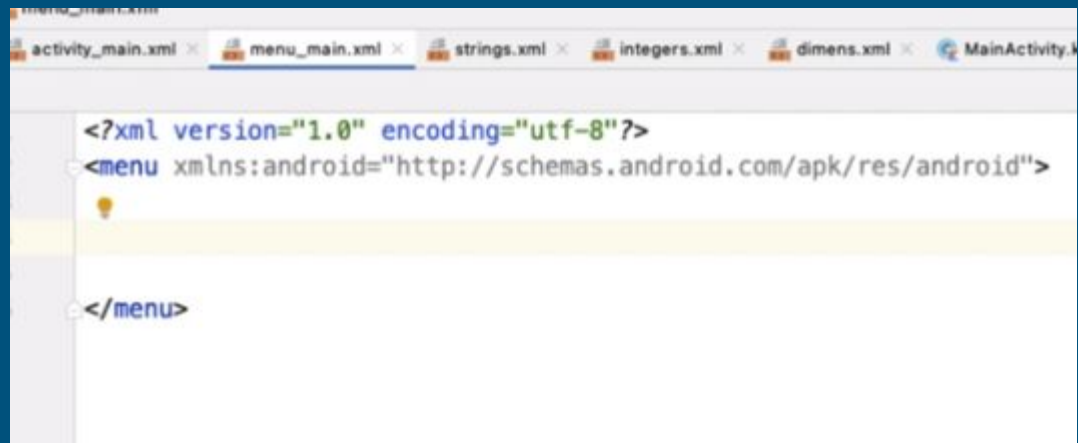
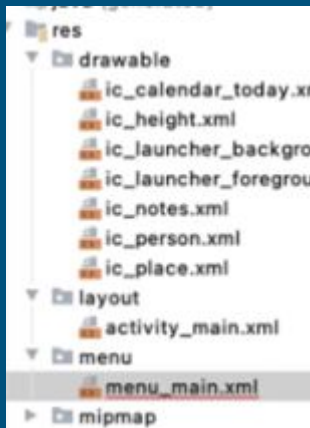


Creamos el resource





Modificamos el menú con un solo elemento

Elemento simple

```
<item  
    android:id="@+id/action_send"  
    android:title="Enviar" ~
```

13:00



Menus&fragments



Enviar



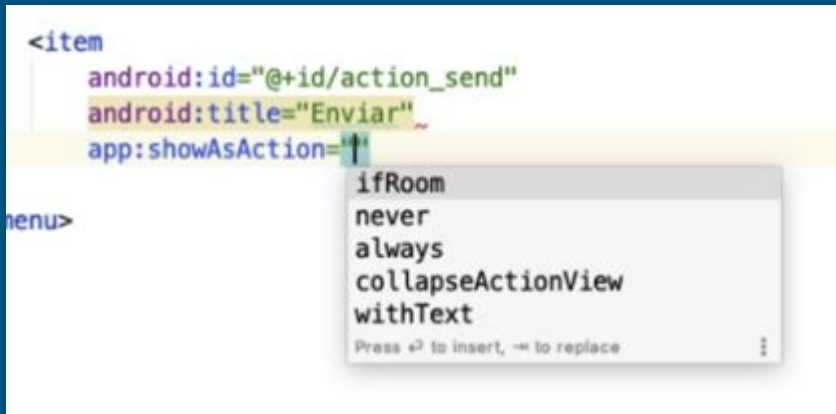
Como queremos mostrarlo

app:showAsAction

always: Muestra el ítem en la barra de acción siempre que haya espacio.

ifRoom: Muestra el ítem en la barra de acción si hay espacio, si no, lo coloca en el menú desplegable.

never: Coloca el ítem en el menú desplegable, incluso si hay espacio.

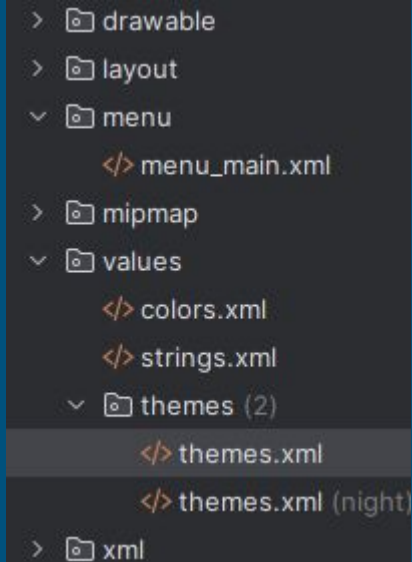


Ahora tenemos que inflar el menu

Infla el menú en tu
Activity o Fragment:

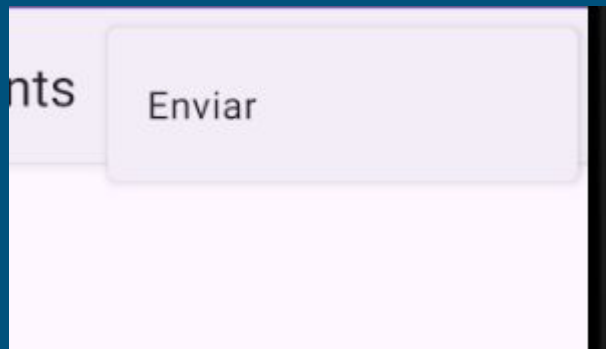
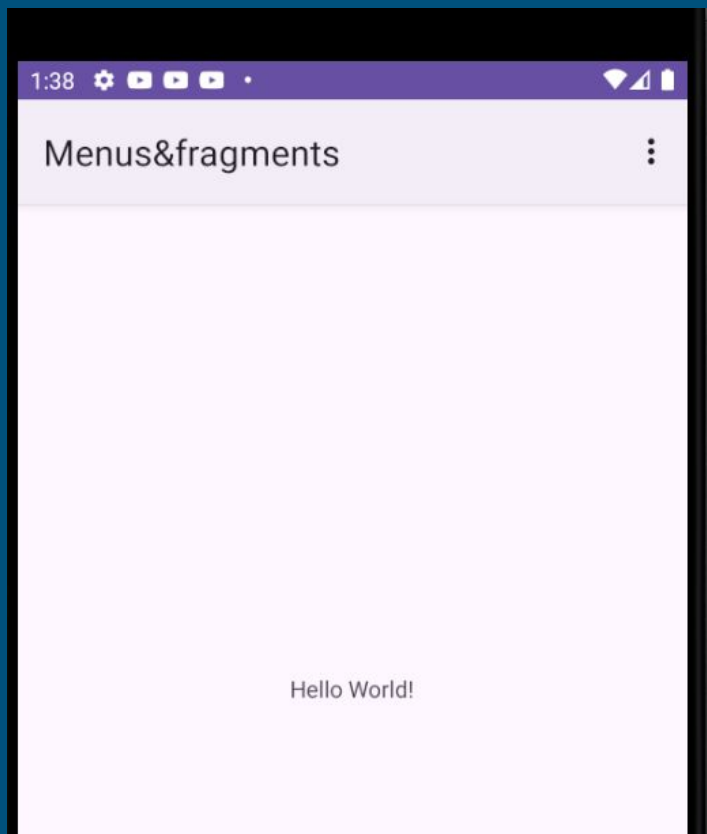
```
activity_main.xml MainActivity.kt menu_main.xml build.gradle
1 package com.tdam.menusfragments
2
3 > import ...
4
5
6
7 </> class MainActivity : AppCompatActivity() {
8     override fun onCreate(savedInstanceState: Bundle?) {
9         super.onCreate(savedInstanceState)
10        setContentView(R.layout.activity_main)
11    }
12
13     override fun onCreateOptionsMenu(menu: Menu?): Boolean {
14         menuInflater.inflate(R.menu.menu_main, menu)
15         return true
16     }
17 }
```

Configuración para actionBar



```
resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Menufragments" parent="Theme.Material3.DayNight">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.Menufragments" parent="Base.Theme.Menufragments" />
</resources>
```



Si queremos mostrarlo

```
<item
  android:id="@+id/actionSend"
  android:title="Enviar"
  android:icon="@drawable/baseline_send_24"
  app:showAsAction="ifRoom"
>

</item>

</menu>
```

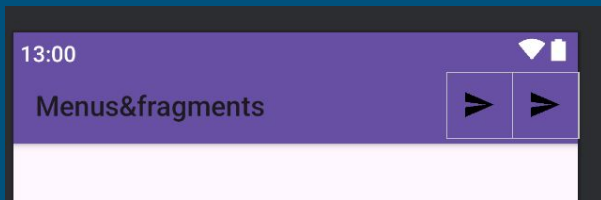
13:00

Menus&fragments



Cómo detectar evento de click

Sobrecargamos la función `onOptionsItemSelected` para detectar qué elemento es pulsado porque podríamos tener más de uno.



```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/actionSend"
        android:title="Enviar"
        android:icon="@drawable/baseline_send_24"
        app:showAsAction="ifRoom"
    ></item>

    <item
        android:id="@+id/actionSendTemporal"
        android:title="Enviar 2"
        android:icon="@drawable/baseline_send_24"
        app:showAsAction="ifRoom"
    >
</item>
```

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
  
    return super.onOptionsItemSelected(item)  
}
```

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
  
    if(item.itemId == R.id.actionSend)  
        //Toast enviar  
        Toast.makeText(context: this, text: "Enviando...", Toast.LENGTH_SHORT).show()  
    else if(item.itemId == R.id.actionSendTemporal){  
        //Toast enviar temporal  
        Toast.makeText(context: this, text: "Enviando temporal...", Toast.LENGTH_SHORT).show()  
    }  
  
    return super.onOptionsItemSelected(item)  
}
```

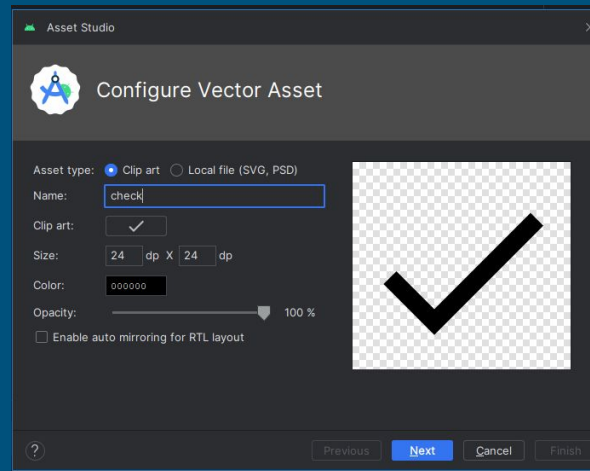
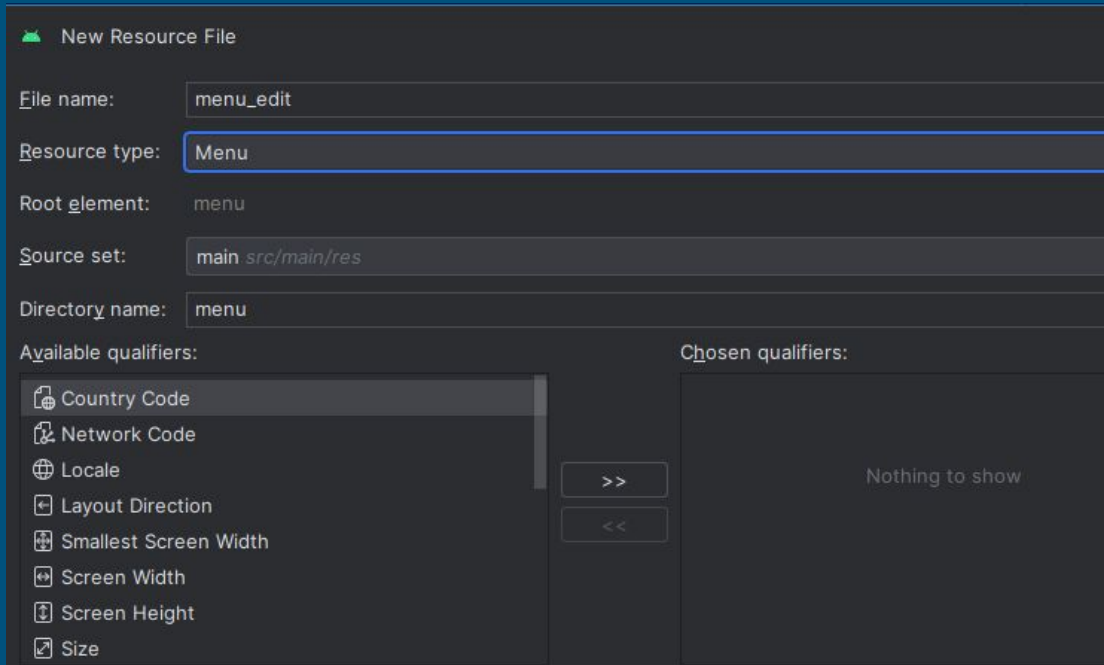
Diseño más complejo

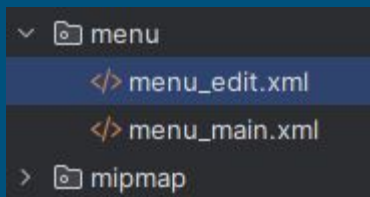
Volvemos a un estado inicial

```
> class MainActivity : AppCompatActivity() {  
  override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  }  
  
  override fun onCreateOptionsMenu(menu: Menu?): Boolean {  
    menuInflater.inflate(R.menu.menu_main, menu)  
    return true  
  }  
  
  override fun onOptionsItemSelected(item: MenuItem): Boolean {  
  
    return super.onOptionsItemSelected(item)  
  }  
}
```

```
<?xml version="1.0" encoding="utf-8"?>  
<menu xmlns:android="http://schemas.android.com/apk/res/android"  
      xmlns:app="http://schemas.android.com/apk/res-auto">  
  
  <item  
    android:id="@+id/action_edit"  
    android:title="Editar datos de usuario"  
    android:icon="@drawable/baseline_send_24"  
    app:showAsAction="never" />  
  
</menu>
```

Creamos el menú de edit





```
activity_main.xml  MainActivity.kt  menu_edit.xml  themes.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <menu xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto">
4
5      <item
6          android:id="@+id/action_save"
7          android:title=""
8          android:icon="@drawable/check"
9          app:showAsAction="ifRoom" />
10
11  </menu>
```

Comprobamos click en la main

```
MainActivity.kt x <> activity_main_edit.xml MainActivityEdit.kt <> menu_edit.xml <> themes.xml <> mer
1 package com.tdam.menusfragments
2
3 > import ...
9
10 <> class MainActivity : AppCompatActivity() {
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContentView(R.layout.activity_main)
14     }
15
16     override fun onCreateOptionsMenu(menu: Menu?): Boolean {
17         menuInflater.inflate(R.menu.menu_main, menu)
18         return true
19     }
20
21     override fun onOptionsItemSelected(item: MenuItem): Boolean {
22
23         if(item.itemId == R.id.action_edit){
24             //Aquí haremos lo que queramos hacer en nuestra accion o app
25
26             //en este caso iniciamos la de editar
27             startActivity(Intent( packageContext.this, MainActivityEdit::class.java))
28         }
29
30
31         return super.onOptionsItemSelected(item)
32     }
33
34     private fun startActivity(java: Class<MainActivityEdit>) {
35
36     }
37 }
```

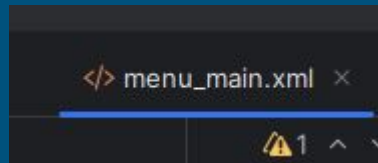
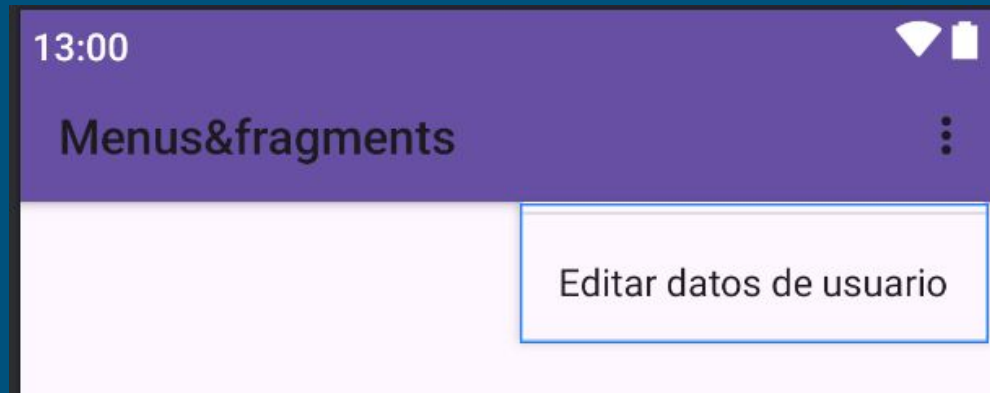
Nuestra primera actividad

Se verá así

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

  <item
    android:id="@+id/action_edit"
    android:title="Editar datos de usuario"
    android:icon="@drawable/baseline_send_24"
    app:showAsAction="never" />

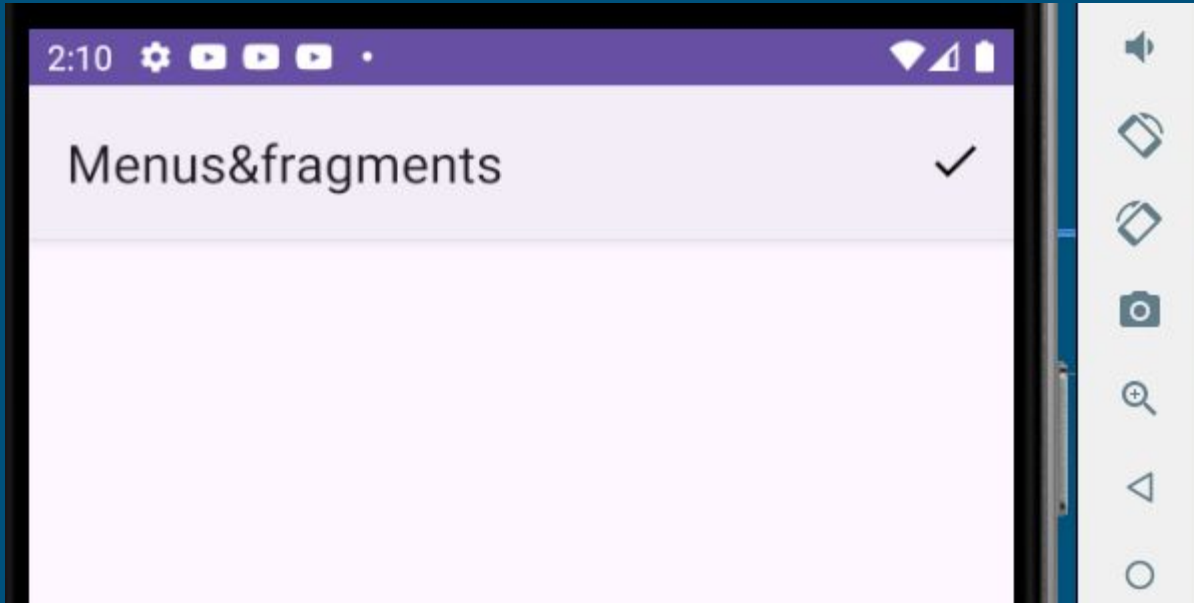
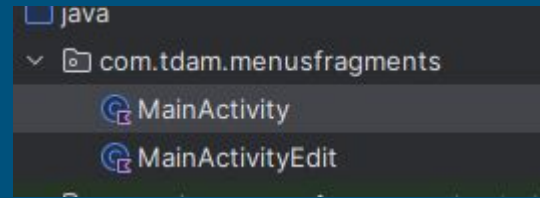
</menu>
```



Comprobamos click en actividad de editar

```
MainActivity.kt  </> activity_main_edit.xml  MainActivityEdit.kt  </> menu_edit.xml  </> themes.xml  </>
1  package com.tdam.menusfragments
2
3  > import ...
4
5
6
7
8  </> class MainActivityEdit : AppCompatActivity() {
9      override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         setContentView(R.layout.activity_main_edit)
12     }
13
14     override fun onCreateOptionsMenu(menu: Menu?): Boolean {
15         menuInflater.inflate(R.menu.menu_edit, menu)
16         return true
17     }
18
19     override fun onOptionsItemSelected(item: MenuItem): Boolean {
20
21         if(item.itemId == R.id.action_save){
22             //Aqui haremos lo que queramos hacer
23             if(item.itemId == R.id.action_save){
24                 //Aqui haremos lo que queramos hacer
25
26                 //luego podemos finalizar la actividad actual y volver a la anterior
27                 finish() //se destruye
28             }
29         }
30     }
31
32     return super.onOptionsItemSelected(item)
33 }
34 }
```

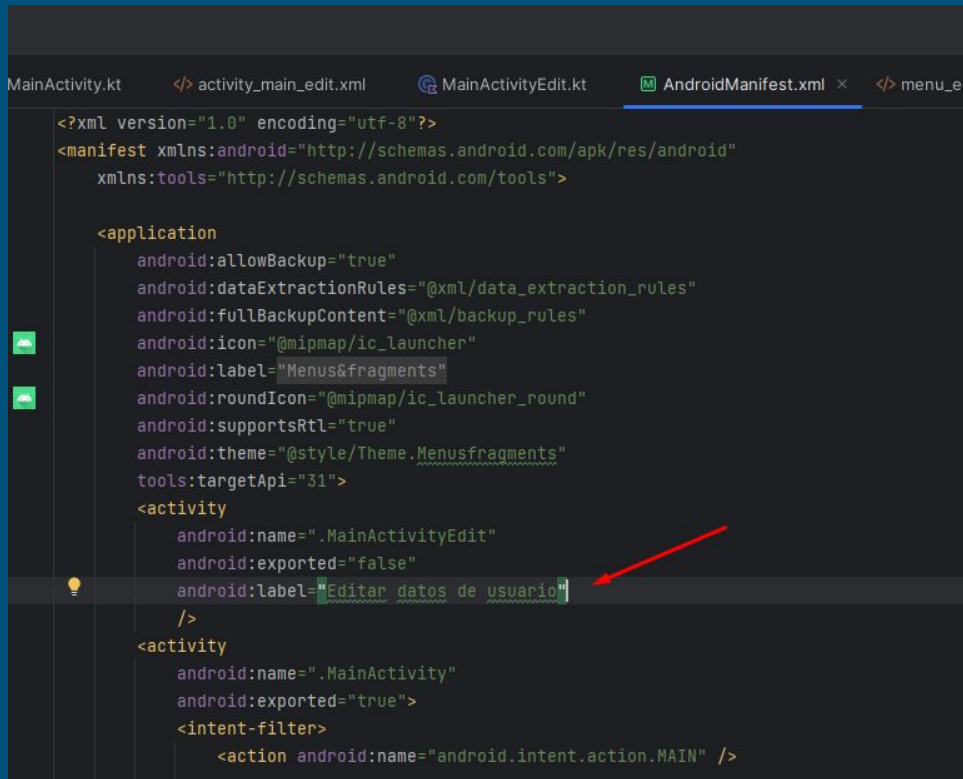
Nuestra primera actividad



Cambiamos el título

Cambiamos el título de la actividad edit para que no tenga el mismo título.

Atributo **label**



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Menufragments"
        tools:targetApi="31">

        <activity
            android:name=".MainActivityEdit"
            android:exported="false"
            android:label="@string/edit_user_data" />

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Ejercicio 1

Crear una vista para que muestre el perfil de un usuario ficticio y otra vista con actividad para poder editar esos datos.

Haz uso de menús para moverte entre actividades.

No hace falta que hagas la funcionalidad de editar aún.

Fragments

¿Qué es un Fragment?

Los Fragments son componentes modulares de interfaz de usuario en Android, diseñados para representar comportamientos o partes específicas de la interfaz en una Activity.

Permiten una estructuración más modular y reutilizable de la interfaz de usuario.

Únicamente pueden pertenecer a una actividad

Ventajas Clave de Fragments

- **Reutilización de UI:** Fragments posibilitan la construcción de componentes de interfaz de usuario independientes y reutilizables en diferentes partes de la aplicación.
- **Diseño Adaptable:** Mejoran la experiencia del usuario al adaptarse a diferentes tamaños de pantalla, como en tablets o dispositivos móviles.
- **Composición Compleja:** Facilitan la creación de interfaces de usuario complejas y estructuradas.

Fragment especial

Crearemos un fragment especial que tiene comportamientos y elementos ya creados para poder hacer más fácil algunas cosas.

En este caso será un fragment para preferences de tipo **SettingsFragment**

La principal diferencia entre un **SettingsFragment** y un fragmento **normal** en Android radica en su propósito y uso específico.

Mientras que un fragmento normal puede utilizarse para cualquier interfaz de usuario en una aplicación, un SettingsFragment está diseñado específicamente para mostrar y gestionar las **preferencias** de **configuración** de una aplicación.

Propósito Específico:

Fragmento Normal: Puede utilizarse para cualquier interfaz de usuario en la aplicación.

SettingsFragment: Está diseñado específicamente para albergar las preferencias y configuraciones de la aplicación.

Interfaz de Usuario Estándar:

Fragmento Normal: Puede contener cualquier diseño de interfaz de usuario personalizado.

SettingsFragment: Sigue las convenciones y directrices de diseño estándar para pantallas de configuración en Android, lo que incluye la presentación de opciones de preferencias, categorías y elementos de interfaz específicos para ajustes.

Uso con Preference API:

Fragmento Normal: Puede gestionar las preferencias manualmente mediante vistas y lógica personalizada.

SettingsFragment: A menudo se utiliza en combinación con la API de preferencias de Android, que proporciona componentes predefinidos y una interfaz más sencilla para gestionar y mostrar las preferencias de la aplicación.

Integración con Preferences Framework:

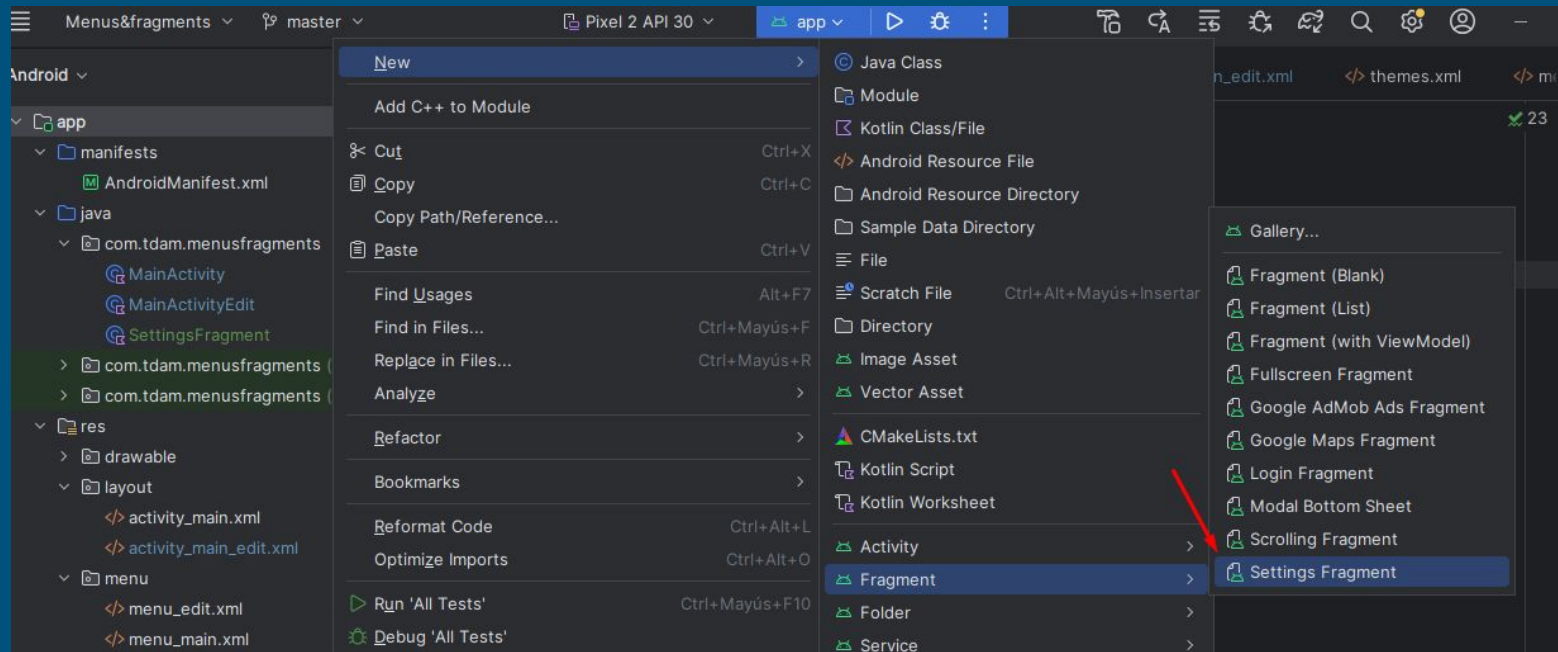
Fragmento Normal: No está necesariamente vinculado al framework de preferencias de Android.

SettingsFragment: Se integra bien con el framework de preferencias de Android, lo que facilita la gestión y visualización de preferencias comunes, como interruptores, cuadros de texto, listas, etc.

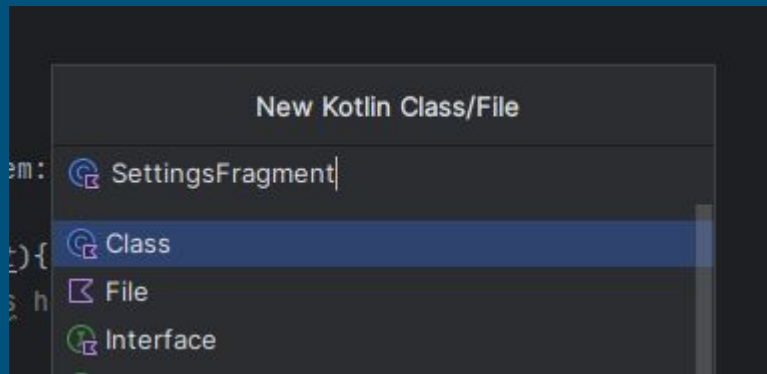
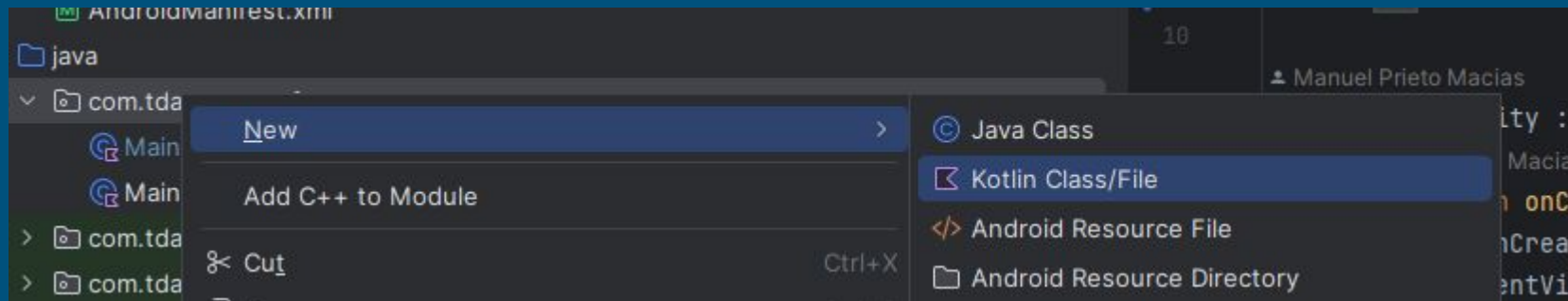
En resumen, un **SettingsFragment** es una especialización de un fragmento diseñada específicamente para alojar y gestionar las preferencias de configuración de una aplicación de manera coherente con las prácticas recomendadas de diseño de Android.

Es una forma conveniente y estandarizada de implementar pantallas de configuración en una aplicación.

Todo esto se puede hacer automáticamente



Creamos un fragment



Cuidado con los antiguos fragments

Siempre usaremos to´ lo de compat para ir a la última

```
1 package com.tdam.menusfragments
2
3 import android.preference.PreferenceFragment
4
5 new *
6 class SettingsFragment : PreferenceFragmentCompat {
7 }
8 }
```

- ⚠ Add dependency on androidx.preference:preference-ktx and import
- ⚠ Create class 'PreferenceFragmentCompat'
- ⚠ Create interface 'PreferenceFragmentCompat'
- ⚠ Create type parameter 'PreferenceFragmentCompat' in class 'SettingsFragment'

```
dependencies {
    implementation("androidx.core:core-ktx:1.9.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("com.google.android.material:material:1.11.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")
    implementation("androidx.preference:preference-ktx:1.2.1")
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}
```

```
package com.tdam.menusfragments

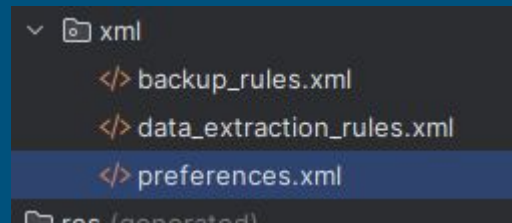
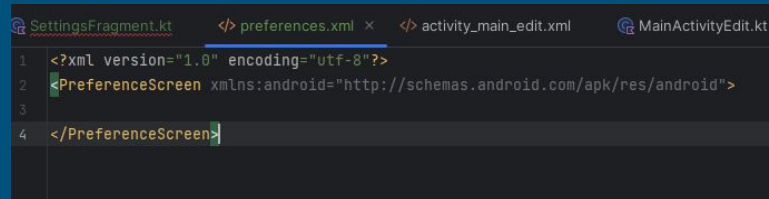
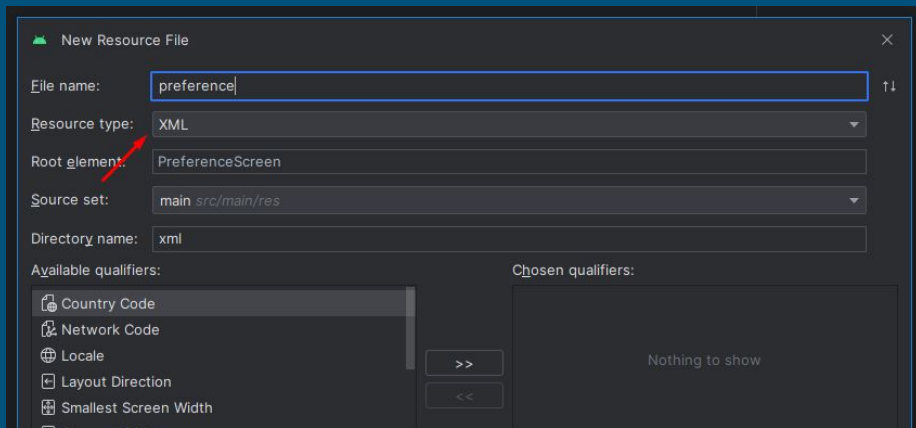
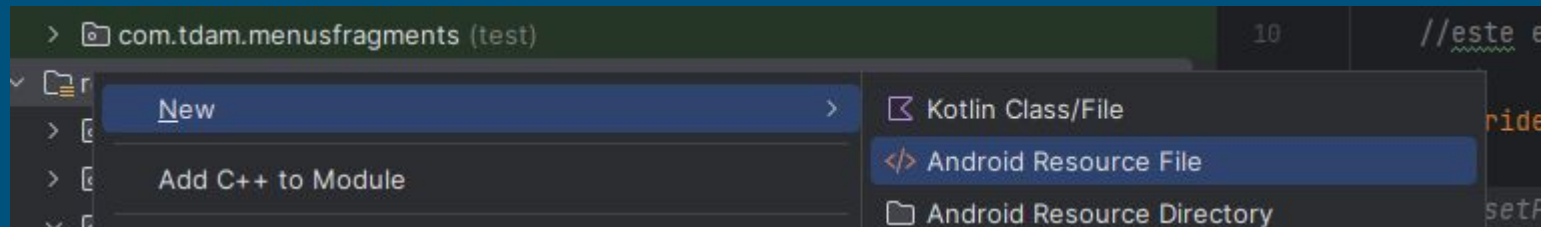
import android.os.Bundle
import androidx.preference.PreferenceFragmentCompat

new *
class SettingsFragment : PreferenceFragmentCompat() {
    new *
}
```

Creamos nuestro OnCreate de fragment

```
5
6 new *
7 class SettingsFragment : PreferenceFragmentCompat() {
8
9
10 //este es el metodo que se encarga de cargar el xml de preferencias
11 new *
12 override fun onCreatePreferences(savedInstanceState: Bundle?, rootKey: String?) {
13     setPreferencesFromResource(R.xml.preferences, rootKey)
14 }
15
16
17 }
```

Ahora su vista



Ahora vinculamos esa vista con nuestro fragment

```
//este es el metodo que se encarga de cargar el xml de preferencias
new *
override fun onCreatePreferences(savedInstanceState: Bundle?, rootKey: String?) {
    setPreferencesFromResource(R.xml.preferences, rootKey)
}
```

Ahora crearemos la “flechita pa’tras”

```
Manuel Prieto Macias *
> class MainActivityEdit : AppCompatActivity() {
    Manuel Prieto Macias *
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main_edit)

        supportActionBar?.setDisplayHomeAsUpEnabled(true) //esto es para que aparezca la flecha de atras
        supportActionBar?.title = "Editar" //esto es para cambiar el titulo de la barra de accion
    }
}

Manuel Prieto Macias
> override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate(R.menu.menu_edit, menu)
```

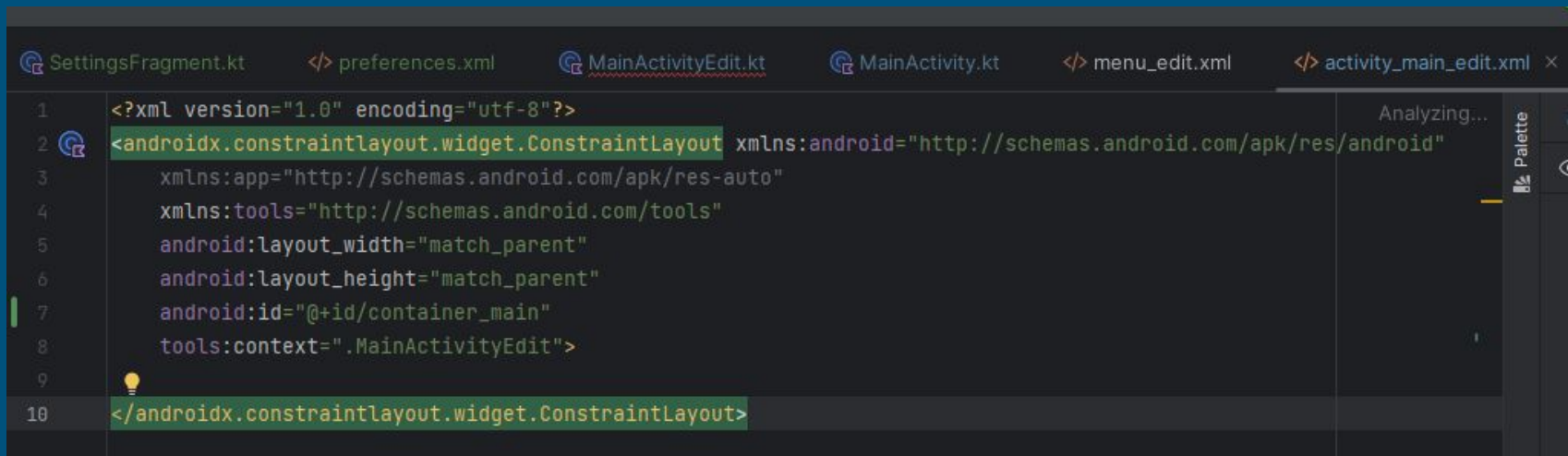
Añadimos el fragment a la actividad

Nos pedirá un
identificador

Vamos a la actividad que
lo usará y le ponemos una
id al layout principal para
usarlo

```
supportFragmentManager  
    .beginTransaction().replace(R.id.  
}
```

```
layout  
    </> activity_main.xml  
    </> activity_main_edit.xml  
    menu
```



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     android:id="@+id/container_main"
8     tools:context=".MainActivityEdit">
9
10 </androidx.constraintlayout.widget.ConstraintLayout>
```

También cambiamos el `constraintLayout` por un `frameLayout` que es mas ligero y está mas enfocado en los `fragments` ya que el `constraintLayout` es innecesario en un fragment.

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
```

Añadimos el id del layout y el fragment que queremos colocar en ese layout

```
supportFragmentManager  
    .beginTransaction().replace(R.id.container_main, SettingsFragment())  
}
```

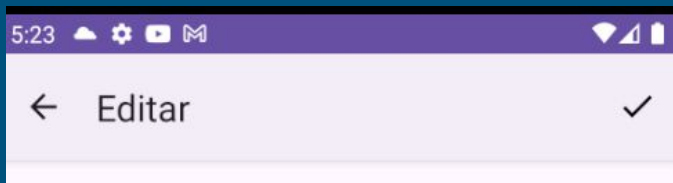
Aplicamos los cambios con **commit()**

```
supportFragmentManager  
    .beginTransaction()  
    |.replace(R.id.container_main, SettingsFragment())  
    .commit()  
}
```

Ahora para finalizar

Sobreescribir el método `onOptionsItemSelected()` para que pueda retroceder el boton de atras.

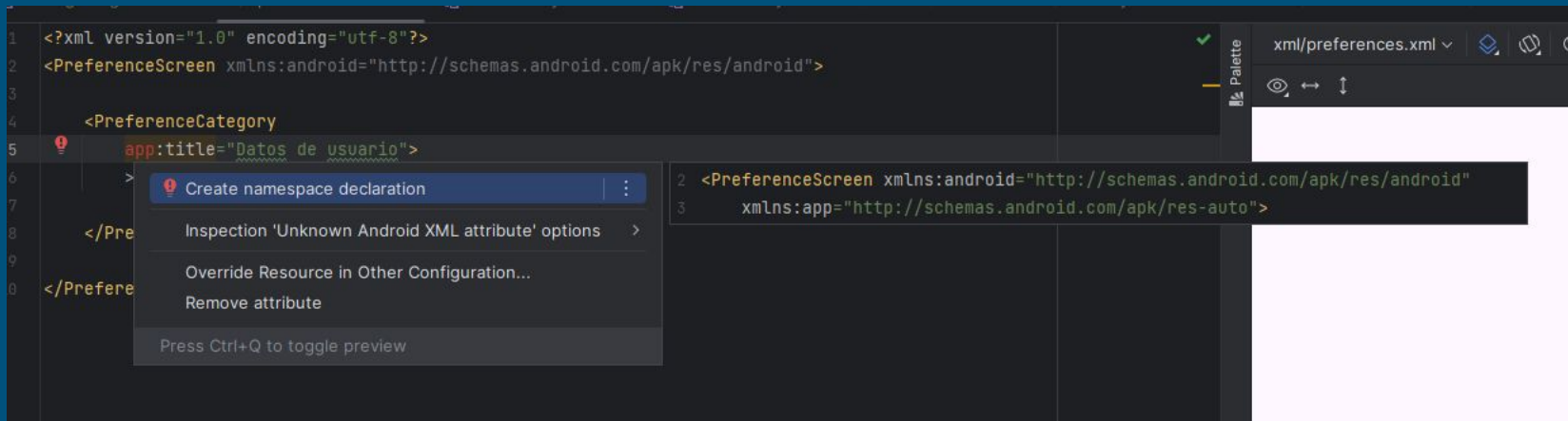
```
Manuel Prieto Macias *  
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
  
    if(item.itemId == android.R.id.home){ //si el id del item es el de la flecha de atras  
        onBackPressedDispatcher.onBackPressed() //se ejecuta la funcion de atras  
    }  
}
```



Diseñamos el fragment

Crearemos una categoría para poder alojar hijos.

Cuidado porque habrá que incluir la librería **app**



Explicaciones

<PreferenceScreen> : Esta etiqueta define un contenedor para las preferencias en la pantalla. Todas las preferencias y categorías deben estar contenidas dentro de esta etiqueta.

<PreferenceCategory> : Esta etiqueta define una categoría dentro de la pantalla de preferencias. Las categorías se utilizan para agrupar y organizar las preferencias relacionadas.

Aclaraciones

Puedes usar nombres diferentes en lugar de "Preference" según tus preferencias y necesidades específicas. La elección de nombres es una cuestión de claridad y coherencia en tu código y diseño.

Yo he usado preferences porque voy a hacer un fragment para guardar datos en preferences

Ejemplo

```
<?xml version="1.0" encoding="utf-8"?>
<UserSettingsScreen xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

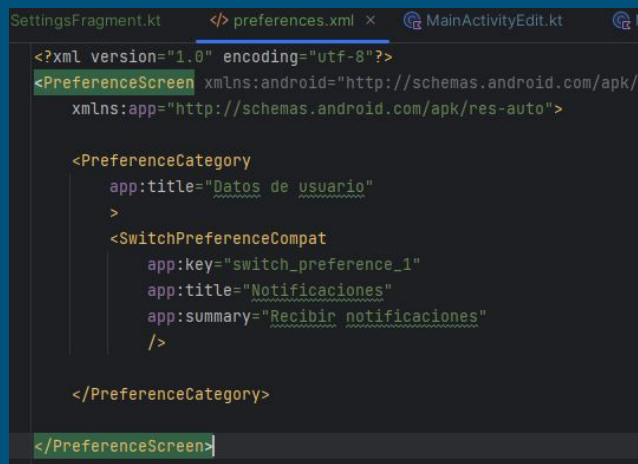
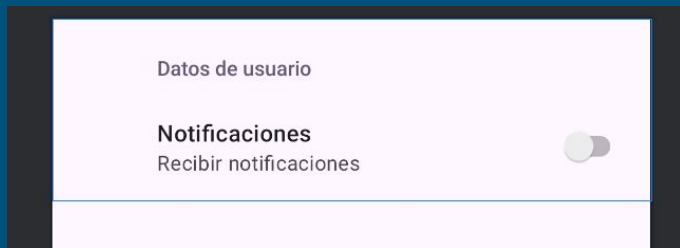
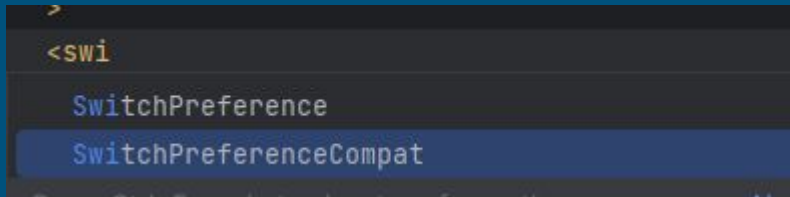
    <UserSettingsCategory
        app:title="Datos de usuario">
    </UserSettingsCategory>

</UserSettingsScreen>
```

Añadimos un hijo de tipo Switch

Usaremos el compat para promover la compatibilidad con distintas versiones de Android. Este elemento servirá para guardar info en sharedPreferences o donde queramos.

La key será usada para vincular el elemento con el valor de la preference.



Le ponemos valor por defecto

```
SettingsFragment.kt  </> preferences.xml  x  MainActivityEdit.kt
1  <?xml version="1.0" encoding="utf-8"?>
2  <PreferenceScreen xmlns:android="http://schemas.android.
3      xmlns:app="http://schemas.android.com/apk/res-auto">
4
5      <PreferenceCategory
6          app:title="Datos de usuario"
7      >
8          <SwitchPreferenceCompat
9              app:key="switch_preference_1"
10             app:title="Notificaciones"
11             android:defaultValue="true"
12             app:summary="Recibir notificaciones"
13             />
14
15      </PreferenceCategory>
16
17  </PreferenceScreen>
```

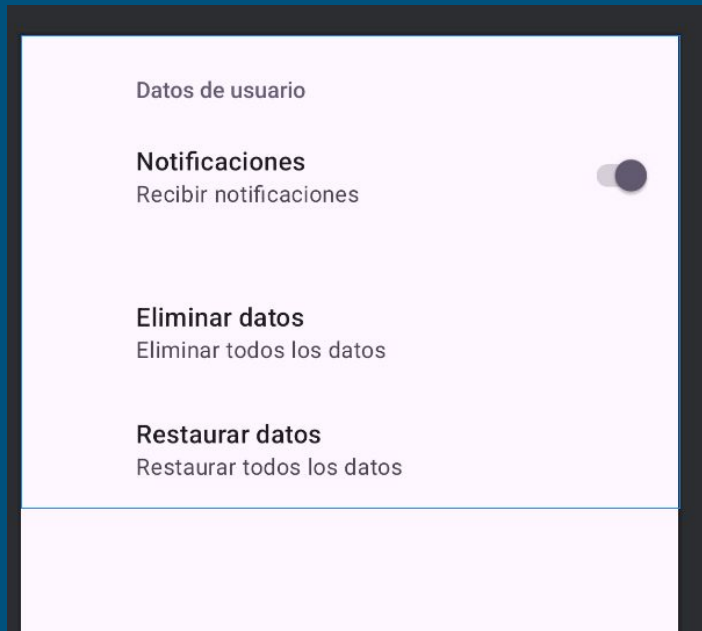
Datos de usuario

Notificaciones

Recibir notificaciones



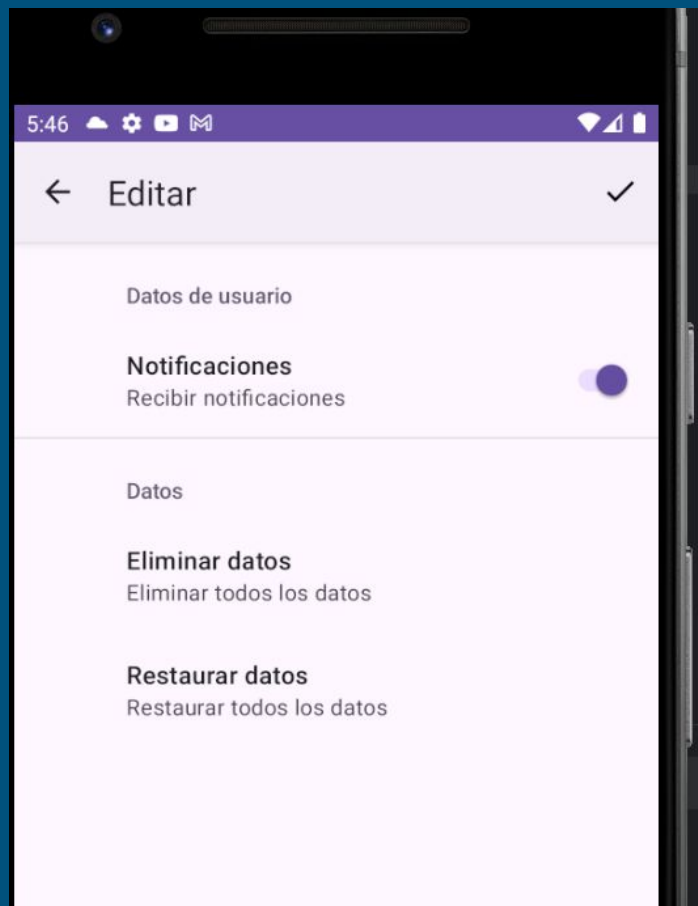
Añadimos más elementos



```
<PreferenceCategory
    app:title="Datos"
>
    <Preference
        app:title="Eliminar datos"
        app:summary="Eliminar todos los datos"
        android:key="eliminar_datos"
    />
    <Preference
        app:title="Restaurar datos"
        app:summary="Restaurar todos los datos"
        android:key="restaurar_datos"
    />

</PreferenceCategory>
```

Ole



Ejercicio 1

Diseñar una interfaz de usuario con dos actividades, una para editar la info de un usuario y otra para mostrarla.

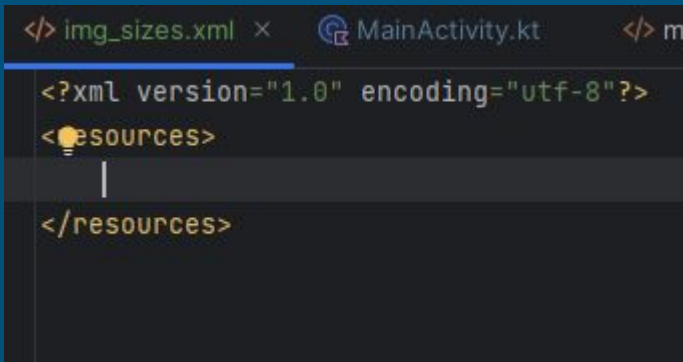
La segunda actividad será un fragment de tipo preference y hará uso de lo visto hasta ahora.

Ejercicio 2

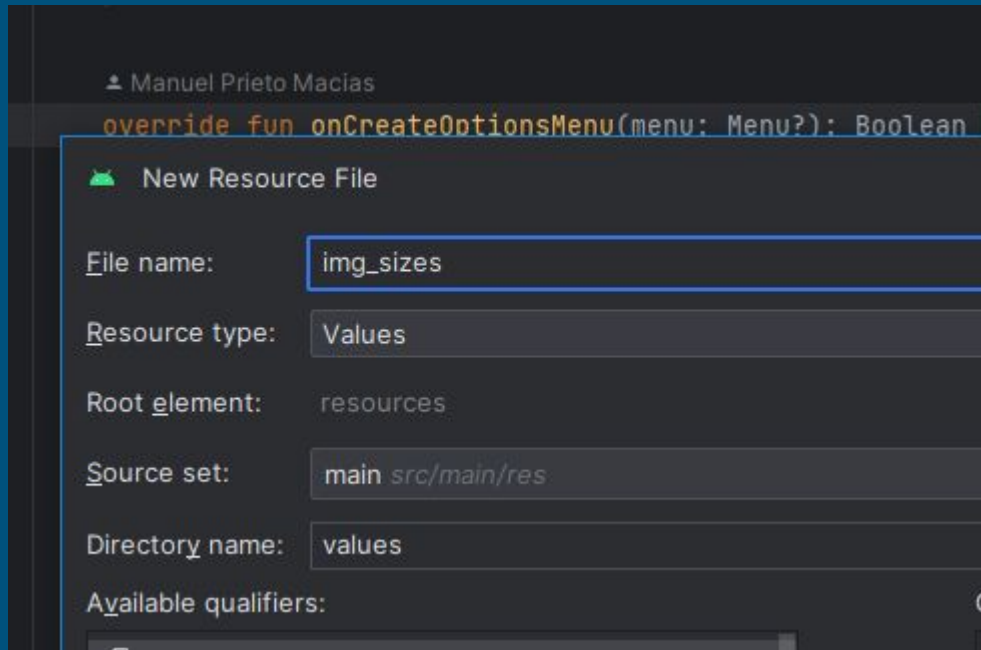
Crear un fragment especial de otro tipo y mostrarlo.

Vamos a añadir otro ítem

Haremos una configuración para decidir si el usuario quiere tamaños de imagen chicos, medianos o grandes.

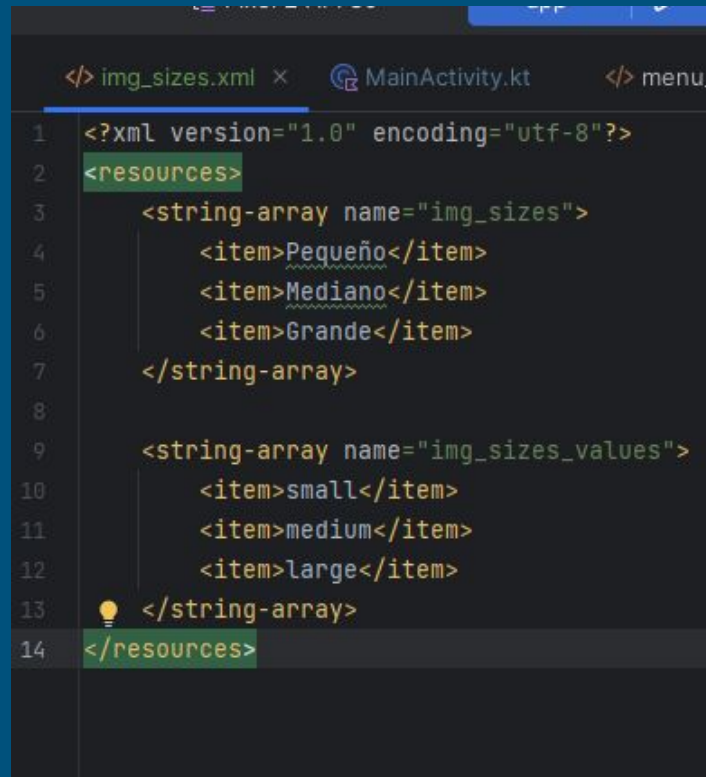


```
<?xml version="1.0" encoding="utf-8"?>
<resources>
|
</resources>
```



Crearemos dos arrays de datos

Uno para la info y otro para los valores de nuestras preferencias.



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3     <string-array name="img_sizes">
4         <item>Pequeño</item>
5         <item>Mediano</item>
6         <item>Grande</item>
7     </string-array>
8
9     <string-array name="img_sizes_values">
10        <item>small</item>
11        <item>medium</item>
12        <item>large</item>
13    </string-array>
14 </resources>
```

Ahora añadimos el elemento de la preference

```
<PreferenceCategory
  app:title="Tamaños imagen"
>
  <ListPreference
    app:key="list_preference_tamamos"
    app:title="Tamaño imagen"
    app:summary="Tamaño de las imágenes"
    app:entries="@array/img_sizes"
    app:entryValues="@array/img_sizes_values"
    android:defaultValue="small"
  />
</PreferenceCategory>
```

app:key: Define una clave única para esta preferencia, que se utilizará para identificarla en el código de la aplicación.

app:title: Es el título que se muestra al usuario en la pantalla de preferencias. En este caso, "Tamaño imagen".

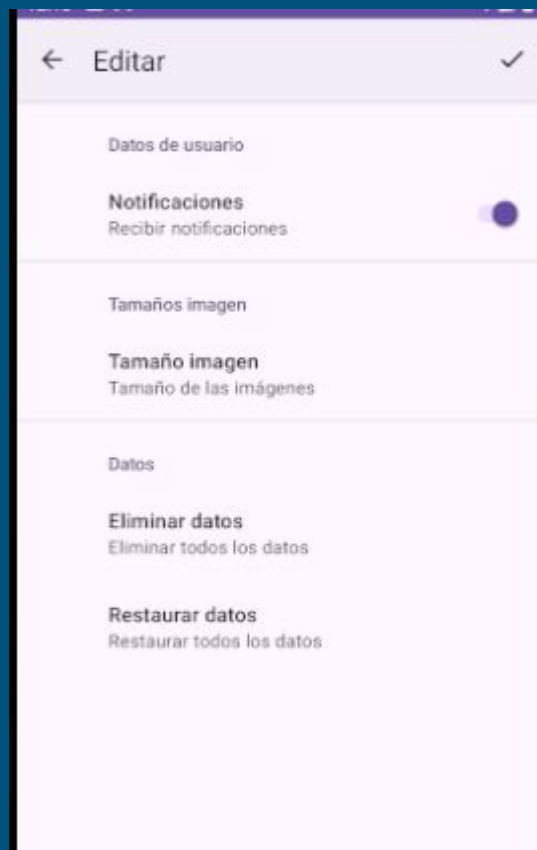
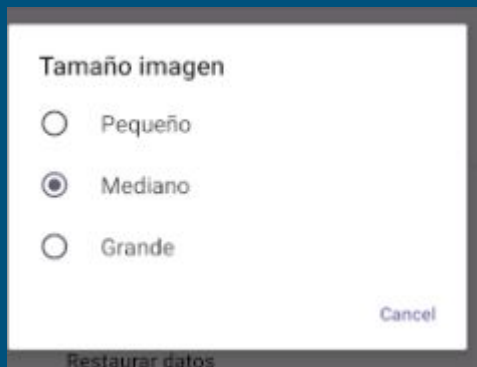
app:summary: Proporciona un resumen o descripción adicional de la preferencia. Aquí, indica que es el "Tamaño de las imágenes".

app:entries: Hace referencia a un conjunto de recursos de cadena (@array/img_sizes) que contiene las etiquetas visibles para el usuario en la lista de preferencias. Estas son las opciones que el usuario verá.

app:entryValues: Se asocia con un conjunto de recursos de cadena (@array/img_sizes_values) que contiene los valores internos asociados a cada opción. Estos valores son los que se almacenan cuando el usuario elige una opción.

android:defaultValue: Especifica el valor predeterminado de la preferencia, en este caso, "small".

Como veis nos crea un alert dialog sin que tengamos que hacer codigo.



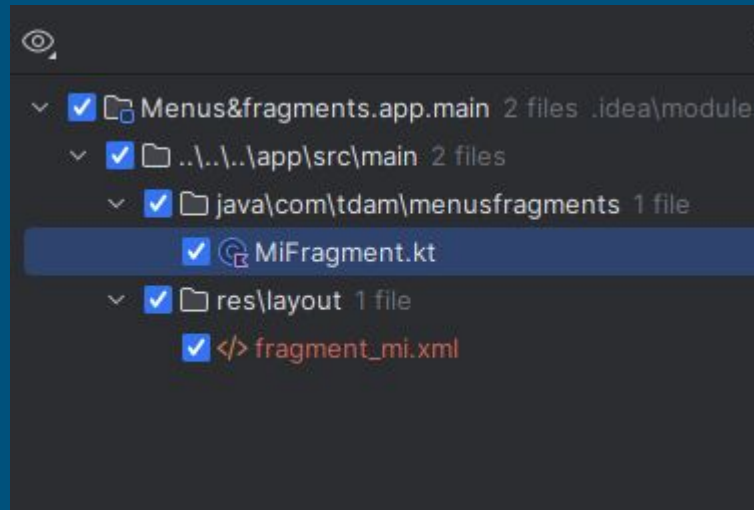
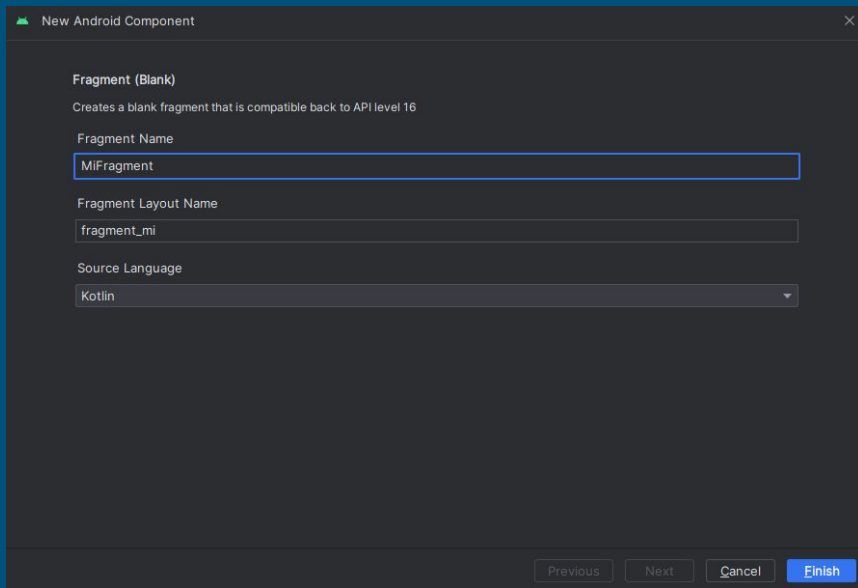
Ejercicio 3

Modificar el código del ejercicio anterior para que al volver del fragment de configuración, la pantalla de usuario muestre las imagen de perfil del usuario como se ha configurado en la preference.

Fragment normal

Paso 1: Crear el Fragmento

Crear una clase para el Fragmento que extienda Fragment.



Paso 2: Implementar el Código del Fragmento

```
new *
```

```
class MiFragment : Fragment() {
```

```
    override fun onCreateView(  
        inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?  
    ): View? {  
        // Inflate the layout for this fragment  
        return inflater.inflate(R.layout.fragment_mi, container, attachToRoot: false)  
    }  
}
```

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<FrameLayout xmlns:android="http://schemas.android.com/ap
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context=".MiFragment">
```

```
    <!-- TODO: Update blank fragment layout -->
```

```
    <TextView
```

```
        android:layout_width="match_parent"
```

```
        android:layout_height="match_parent"
```

```
        android:text="Hello blank fragment" />
```

```
</FrameLayout>
```

Paso 3: Utilizar el Fragmento en una Actividad

Agrega el fragmento al diseño de la actividad mediante el contenedor `FrameLayout`.

```
<!-- activity_main.xml -->  
<FrameLayout android:id="@+id/contenedorFragmento"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    xmlns:android="http://schemas.android.com/apk/res/android" />
```

En el código de la actividad, instancia el fragmento y lo agrega al contenedor.

```
val fragmento = MiFragmento()
supportFragmentManager.beginTransaction()
    .replace(R.id.contenedorFragmento, fragmento)
    .commit()
```

